KAVACH

INCIDENT INTELLIGENCE AGENT — EXACT FLOWCHART

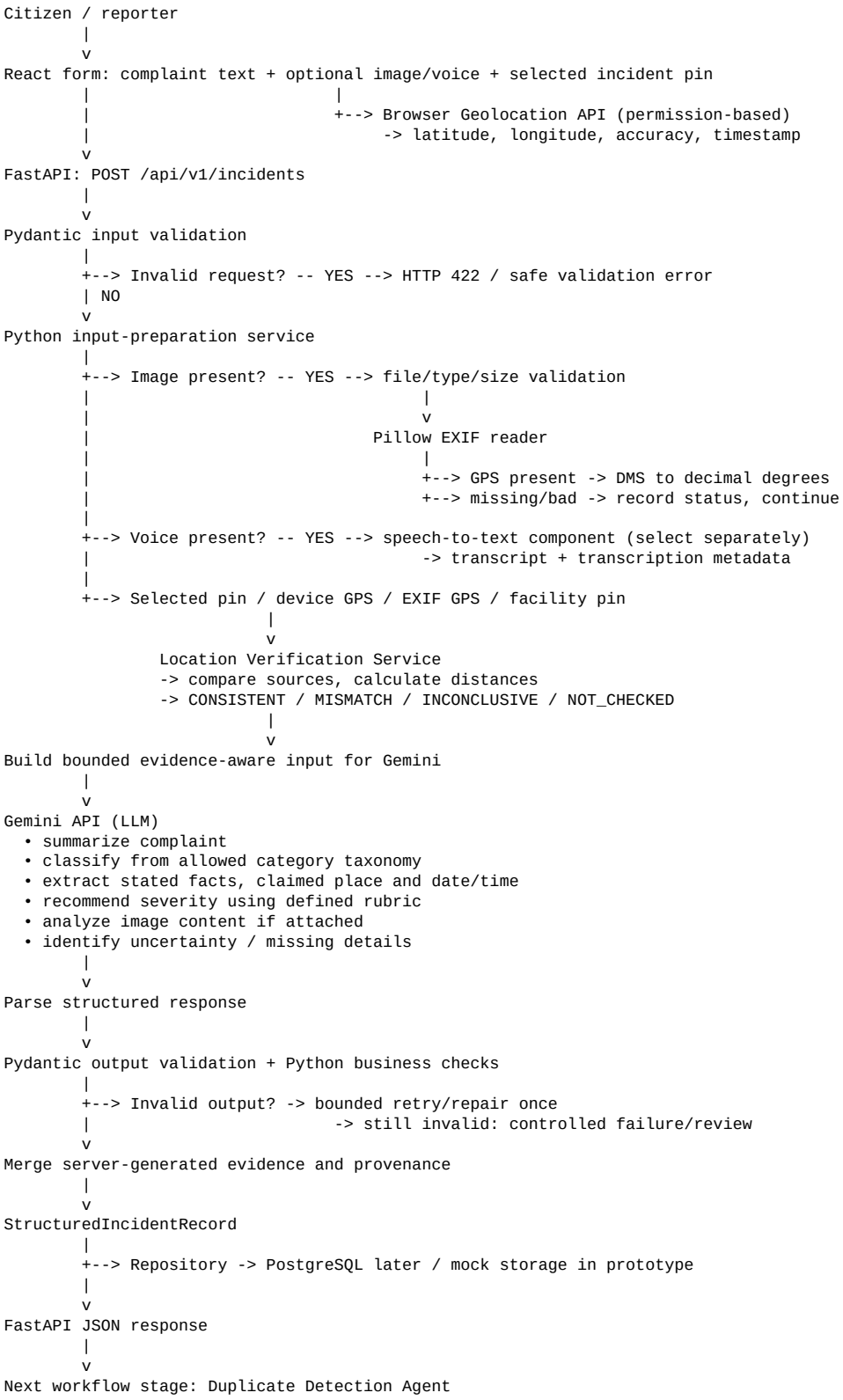
Processing steps • technology at each step • branches • inputs and outputs

Scope: Agent 1 only. The flow begins with complaint submission and ends with a validated structured incident record ready for Duplicate Detection. The architecture below is a proposed design, not a claim that it is already implemented.

Design decision	Chosen responsibility
API entry point	FastAPI receives request and returns response.
Validation	Pydantic validates input and output structure; Python checks business rules.
LLM	Gemini interprets language and optional image content; it does not calculate distance or prove location authenticity.
Image metadata	Pillow reads available EXIF GPS; Python converts/validates coordinates.
Location comparison	A deterministic Location Verification Service compares source coordinates and returns reason codes.
Workflow	Python orchestrates steps; LangGraph is optional if adopted for branching/retries.
Persistence	Repository abstraction; in-memory/mock for isolated prototype, PostgreSQL for persistence.

Version 1.0 • 12 October 2026

1. Master flowchart



Rule: deterministic code handles input validation, EXIF parsing, coordinate math, evidence provenance and schema checks. Gemini handles semantic understanding. The model must not invent missing GPS or timestamp values.

2. Step-by-step: technology and output

Step	Technology	Exact job	Output
1. Collect	React UI	Collect text, optional media, incident pin and consented location	Raw complaint
2. Capture GPS	Browser Geolocation API	Permission-based current device location, accuracy and timestamp	DeviceLocation or unavailable
3. Receive	FastAPI	Receive request, enforce auth/limits, route request	Request context
4. Validate input	Pydantic	Validate types, required fields, coordinate ranges and enums	Valid request or HTTP 422
5. Validate image	Python + Pillow	Safely decode image and inspect EXIF GPS IFD	Metadata result
6. Normalize EXIF	Python converter	Convert DMS to decimal, apply N/S and E/W signs, validate result	Normalized GPS or missing/error
7. Compare location	Location Verification Service	Compare selected pin, device GPS, EXIF and facility reference when available	Status, distance, reasons
8. Build context	Python prompt builder	Provide complaint, taxonomy, severity rubric, permitted image and compact evidence summary	LLM input
9. Interpret	Gemini API	Summarize, classify, extract facts, recommend severity, inspect image	Candidate structured response
10. Validate output	Pydantic + Python	Check schema, enums, lengths and required uncertainty flags	Validated candidate
11. Merge	Python service	Attach server IDs/timestamps and deterministic location result; preserve sources	Incident record
12. Persist	Repository / PostgreSQL	Save incident and evidence references; mock storage is okay for prototype	Record ID
13. Return	FastAPI	Return stable JSON response	API response
14. Continue	Workflow / optional LangGraph	Pass structured record to next stage	Duplicate Detection input

3. Location evidence branch

```
Separate sources:
 A. USER_SELECTED_PIN — where citizen says the incident is
 B. DEVICE_GEOLOCATION — where browser/device reports it is now
 C. IMAGE_EXIF — GPS embedded in image, if present
 D. FACILITY_REFERENCE — approved registered site coordinates

Python validates each source separately:
 - latitude/longitude ranges and finite numbers
 - accuracy where available
 - timestamp and source provenance
 - never silently overwrite one source with another

Distance calculation:
 - Haversine formula for prototype
 - PostGIS optional later for spatial searches

Result:
 - CONSISTENT
 - MISMATCH
 - INCONCLUSIVE
 - NOT_CHECKED
 plus distance_meters, reason_codes, rule_version

Attach result to the incident and pass a compact summary to Gemini.
Do NOT let the LLM declare a complaint fake solely from a GPS mismatch.
```

Situation	Correct handling
Citizen is far away	May be a valid remote report; treat as a signal, not proof of fraud.
Device and EXIF disagree	Preserve both; add SOURCE_CONFLICT and review if policy requires.
EXIF absent	Record EXIF_GPS_MISSING; continue using other evidence.
Accuracy is poor	Return inconclusive or review; do not treat point as exact.
GPS matches target	Location is consistent, but the issue itself is not proven.
Permission denied	Use configured fallback reporting flow.

Recommendation: accept citizen complaints from anywhere. For workers, task-time location can be part of completion evidence, with an exception path.

4. Proposed input and output contracts

Input contract (simplified)

```
IncidentIntelligenceInput
- request_id: str
- complaint_text: str
- selected_incident_location: Coordinates | None
- device_location: DeviceLocation | None
- image_refs: list[MediaReference]
- image_exif_results: list[ImageMetadataResult]  # server-generated
- voice_transcript: str | None
- location_check: LocationCheckSummary | None  # server-generated
- submitted_at: datetime  # server-generated
- taxonomy_version: str
```

Output contract (simplified)

```
StructuredIncidentRecord
- incident_summary: str
- category: allowed enum
- incident_type: allowed enum or UNKNOWN
- severity_recommendation: LOW | MEDIUM | HIGH | UNKNOWN
- severity_reason: str
- extracted_facts: list[ExtractedFact]
- claimed_location_text: str | None
- reported_incident_location: Coordinates | None
- location_evidence_summary: LocationEvidenceSummary
- image_observations: list[str]
- missing_information: list[str]
- conflicting_information: list[str]
- needs_clarification: bool
- clarification_questions: list[str]
- evidence_provenance: list[EvidenceReference]
- created_at: datetime  # server-generated
```

Keep model confidence separate from device accuracy. LLM confidence is not automatically calibrated; explain evidence and uncertainty field-by-field.

5. Suggested repository layout

```
src/kavach/
  api/routes/incidents.py
  schemas/incident_intelligence.py
  schemas/location.py
  services/incident_intelligence/
    orchestrator.py
    prompt_builder.py
    taxonomy.py
    severity_rules.py
    output_validator.py
  services/location/
    exif_reader.py
    coordinate_conversion.py
    distance.py
    verification.py
  services/media/validation.py
  agents/incident_intelligence.py
  repositories/incident_repository.py
  tests/
    unit/test_taxonomy.py
    unit/test_severity_rules.py
    unit/test_exif_reader.py
    unit/test_location_verification.py
    unit/test_output_validation.py
    integration/test_incident_submission.py
```

6. Build sequence

1. Agree incident categories and a severity rubric.

2. Define input/output Pydantic schemas and source/status enums.

3. Implement deterministic coordinate validation, distance and status rules.

4. Implement safe image validation, EXIF reading and DMS conversion.

5. Build Gemini prompt and structured response parsing with mock mode.

6. Validate output, merge server-generated evidence and provenance.

7. Expose FastAPI endpoint and repository abstraction.

8. Test missing GPS, conflicting sources, malformed images, invalid model output and remote citizen reports.

9. Connect to Duplicate Detection only after the agent contract is approved.

7. Error branches and definition of done

Case	Expected behaviour
No image	Continue text-only; image observations empty.
No EXIF GPS	Continue; record missing metadata.
Malformed/oversized image	Safely reject or mark media invalid; do not crash.
GPS permission denied	Allow fallback and mark location unavailable.
Gemini timeout/quota	Controlled retryable failure; mock mode only in development; never fabricate.
Malformed model response	Validate, bounded retry/repair, then controlled failure.
Unknown category	Use OTHER/UNKNOWN and flag uncertainty.
Conflicting sources	Preserve all sources and add reason code.
Citizen far from incident	Do not automatically label complaint fake.
Sensitive logs	Do not log raw EXIF or precise coordinates by default.

Definition of done: valid complaints produce a validated structured record; location sources stay separate; uncertainty is explainable; mock/offline tests pass; the API contract is stable; and downstream responsibilities remain outside this agent.

Implementation references

Geolocation API: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API

Pillow EXIF/Image: https://pillow.readthedocs.io/en/stable/reference/Image.html

FastAPI file uploads: https://fastapi.tiangolo.com/tutorial/request-files/

Pydantic models: https://docs.pydantic.dev/latest/concepts/models/

LangGraph overview (optional): https://docs.langchain.com/oss/python/langgraph/overview